

Universal Trust Substrate (UTS): A Composable Zero-Knowledge Stack for Provable Execution, Memory, and Reasoning

Abstract

The Universal Trust Substrate (UTS) is a **hybrid zero-knowledge stack** designed to provide verifiable integrity and privacy across execution, memory, and reasoning in decentralized systems. UTS combines multiple layers: **zkSyslog** as a verifiable event log, **Zero-Knowledge Graph (zkGraph)** as a structured memory layer, **zkRAG** (zero-knowledge retrieval-augmented generation) as a cognitive reasoning layer, and **Hybrid Proof Execution Infrastructure (HPEI)** blending STARK and SNARK proofs for computation. All proofs are verified on-chain (e.g. on Starknet) for end-to-end trust. This whitepaper presents the UTS architecture in a formal yet accessible manner, detailing how each layer functions and how they compose to enable **infrastructure-level trust**. We illustrate new asset types and autonomous agent models made possible by UTS – from privacy-preserving compliance in DeFi to auditable multi-agent AI systems – and argue that UTS can serve as a **universal substrate for provable execution, memory, and reasoning** in future financial and AI-driven networks.

Introduction and Background

Modern decentralized applications and autonomous agent systems face a fundamental trade-off between **transparency and privacy** in establishing trust. Traditional blockchains provide public verifiability of transactions but at the cost of full data transparency, which is untenable for sensitive logic or data (e.g. private business logic, personal information, or AI agent chain-of-thought). A naive approach might log every internal event or decision on-chain for auditability, but this **“log everything” strategy fails due to privacy loss and data bloat** ¹ ². For example, logging every step of an AI’s reasoning or every detail of a financial transaction on a public ledger would expose sensitive information and quickly overwhelm storage. Yet doing nothing means **trust gaps** remain – agents or protocols could misbehave or lie about their internal actions without detection ³.

Zero-knowledge proofs (ZKPs) have emerged as a breakthrough technology to resolve this dilemma by allowing verifiable claims without revealing underlying data ⁴. In recent years, ZKPs have been applied to blockchains (e.g. Zcash’s private transactions) and even to AI and data queries. For instance, **verifiable computing** techniques enable proving that a computation was executed correctly without re-running it, and researchers have shown it’s feasible to prove database queries and even machine learning inferences in zero-knowledge ⁵ ⁶. Notably, a 2025 milestone saw the full inference of a GPT-2 language model proved in zero-knowledge, **demonstrating that verifiable AI is now a reality** ⁶.

However, a single proof or application is not enough – what’s needed is a **composable stack of trust** that covers the spectrum from low-level events to high-level reasoning. The **Universal Trust Substrate (UTS)**

aims to fill this role. UTS is conceived as a *modular, layered architecture* providing **privacy-preserving verification** at the infrastructure level across four key aspects:

- **Verifiable Event Logging (zkSyslog)** – A reliable, tamper-evident log of events or actions, where integrity and ordering can be proven without exposing event contents.
- **Verifiable Memory/State (zkGraph)** – A structured knowledge graph or state repository for agents and contracts, supporting queries whose correctness can be proven in ZK without revealing the underlying data.
- **Verifiable Reasoning (zkRAG)** – A reasoning layer for AI or complex logic that uses Retrieval-Augmented Generation with provable use of knowledge and adherence to constraints, enabling **cognitive trust** in autonomous decisions.
- **Verifiable Execution (HPEI)** – A hybrid proof system combining **STARK** and **SNARK** technologies to prove general computation execution and any hidden computations, yielding efficient proofs that can be verified on-chain (specifically on Starknet).

All these layers feed into **on-chain verification logic** (smart contracts) that enforce trust by accepting or rejecting actions based on proof verification. Starknet, a ZK-Rollup on Ethereum, is a natural home for this logic due to its low-cost, scalable verification of proofs.

In the following sections, we detail each layer of the UTS stack, its design and role. We then illustrate how UTS enables new asset types and agent models – from **regulated DeFi** instruments that prove compliance without leaking user data, to **multi-agent systems** of AIs that can cooperate under cryptographic assurances of fair play. Throughout, we emphasize **composability** (each layer can integrate with others), **privacy-preserving verification** (proving truths without revealing secrets), and the shift from application-specific trust to **infrastructure-level trust**.

By combining these layers, UTS serves as a **universal substrate for provable execution, memory, and reasoning**. It provides a foundation on which developers and researchers can build systems that are both highly secure/regulated and innovation-friendly. The paper concludes with a thesis on how such a substrate will shape the future of **multi-agent cooperation, regulated decentralized finance, and verifiable AI systems**.

(Diagram – Conceptual Stack Architecture: Envision a layered diagram with UTS components stacked. At the base is On-Chain Verifier (Starknet), above which sits Hybrid Proof Execution (STARK+SNARK). Above that are parallel layers: on one side zkSyslog (Event Log) and zkGraph (Knowledge Graph) providing data to, on the other side, the zkRAG Reasoning Engine. Arrows indicate: events feed into the graph; the graph and external queries feed into reasoning; the reasoning and computations generate proofs via the HPEI; all proofs flow down to the on-chain verifier. The result is a closed trust loop – any statement or action by an agent in this system can be verified by following the chain of proofs down to on-chain verification.)

UTS Architecture and Components

zkSyslog: Verifiable Event Log Layer

The **zkSyslog** layer provides a **cryptographically verifiable audit log** for events, analogous to a system log in traditional computing but enhanced with integrity proofs and privacy. Every action or significant event generated by an agent or protocol is recorded as a log entry that is cryptographically chained and

committed, so that the sequence of events is tamper-evident and each entry can be proven to be part of the log. Crucially, zkSyslog allows proving facts about the log *without* revealing all log contents, addressing the privacy issue of naive logging.

Each log entry (e.g. an AI agent's tool invocation, a transaction step, or a sensor reading) is hashed and linked to the previous entry's hash (forming a hash chain). This provides **append-only integrity** – no entry can be altered or removed without breaking the chain. In addition, entries are periodically aggregated into a **Merkle tree** whose root is published (e.g. on-chain or to a transparency service). As a result, one can produce an **inclusion proof** (a Merkle proof) to show a given event is in the log, or a consistency proof to show the log grew in a certain way ⁷ ⁸. An auditor or smart contract can verify a specific event occurred by checking a short Merkle proof against the known root, **“without revealing any other entries”** in the log ⁹. This means zkSyslog achieves **efficient, privacy-preserving auditability**.

To ensure authenticity (knowing *who* logged the event), each entry can be digitally signed by the agent or component that emitted it. The log thus provides non-repudiation (an agent cannot deny an action it signed) and tamper-evidence (hash linking prevents undetected edits) ¹⁰. The design typically involves an **isolated logging service or sidecar** that agents send events to, preventing a compromised agent from simply deleting its own log entries ¹¹ ¹².

Privacy and Selective Disclosure: zkSyslog does not necessarily put raw event data on-chain. Rather, it allows *commitment to events* off-chain and later selective proofs. For example, an agent could log every function call it makes (with parameters hashed). If a dispute arises (say another agent or a regulator queries a specific action), the agent can reveal just the relevant entry and a proof of inclusion in its log. This way, **agents prove their behavior faithfully without exposing their entire chain-of-thought or operations** ¹. As Rodriguez notes in the context of AI agents, *“we want to verify that resources were actually spent, without exposing each private chain-of-thought... Enter ZK proofs: a cryptographic tool that proves actual expenditures without revealing sensitive details.”* ¹. zkSyslog is the layer enabling that: a public record of *hashes* of events (ensuring accountability) combined with zero-knowledge techniques to reveal only what's needed.

Example: Imagine a high-frequency trading bot operating off-chain. It keeps a zkSyslog of all trades and price data it used. Later, to comply with an audit, it can prove it never exceeded a risk limit or never traded on insider data by providing ZK proofs that certain sequences or patterns **do not appear** in its log (without publishing the log itself). Similarly, in an AI agent network, if one agent challenges another's claimed work, the challenged agent can provide a proof that it indeed performed N steps (logged in its zkSyslog) consistent with its claims ¹³ ¹⁴, or have its stake slashed if it fails to do so ¹⁵. This incentivizes honesty and **stable cooperation** among agents ¹⁶.

Thus, zkSyslog serves as the **ground truth event ledger** in UTS. It lays the foundation for trust by ensuring every low-level event can be independently verified for integrity and inclusion. The subsequent layers – memory and reasoning – will build on this stream of verifiable events.

zkGraph: Zero-Knowledge Graph Memory Layer

On top of the raw event log, autonomous systems and protocols need structured memory or state. The **Zero-Knowledge Graph (zkGraph)** layer provides a persistent, queryable knowledge base (which can be thought of as a graph or database of facts) with built-in cryptographic commitments so that any query or

assertion against this memory can be verified. In simpler terms, zkGraph is like a **knowledge graph** or state database that comes with **proofs of correctness** for any information extracted from it – all without revealing the confidential parts of that data.

In UTS, zkGraph can store various structured data: - **Agent knowledge and world state:** e.g. an AI agent's memory of entities and relations, represented as nodes and edges in a graph. - **Contract or protocol state:** e.g. positions, credentials, or conditions in a DeFi protocol that might be too complex or private for a public state. - **Derived facts from events:** the zkSyslog events can be ingested into zkGraph as well, so the graph accumulates state (for instance, a “balance” node updated by deposit/withdraw events, or an “experience” node for an agent updated by past actions).

What makes zkGraph powerful is that it supports **zero-knowledge queries**. A data owner (or an agent holding the private state) can prove to a verifier that a certain query result is correct with respect to the committed graph, *without revealing the underlying data beyond the query answer*. Recent research has demonstrated this is feasible: for example, Wu et al. (2025) introduced *ZKGraph*, a system for verifiable graph query processing where the database owner can prove a query was executed correctly on their graph data **without disclosing the data itself** ⁵ ¹⁷. In formal terms, zkGraph provides **zero-knowledge verifiability on graph queries** – the verifier is convinced that *“the query results are correctly computed by executing the specified operations on the data owner's declared dataset”*, yet the prover *“provides such a proof without disclosing the dataset information.”* ¹⁷.

Structure: The zkGraph could be implemented as a commitment to the entire graph (for example, a Merkle hash of all nodes and edges, or using polynomial commitments for dynamic queries). Each update to the graph (e.g. adding an edge or updating a node's value) would update this commitment. To answer a query (like “Is X connected to Y?” or “What is the value of attribute Z?”), the prover runs the query on their private graph data and then produces a zkSNARK/STARK proof attesting to the correctness of the result. Techniques like circuit decomposition and custom arithmetizations are used to handle graph algorithms efficiently in ZK ¹⁸ ¹⁹ – for instance, breaking a complex graph query into smaller primitive steps that each have manageable proof circuits (as raw graph traversal can be expensive to prove directly).

Use Cases: In a multi-agent UTS scenario, an agent could use zkGraph to represent its *beliefs* or *knowledge*. If one agent needs to trust another's claim (“I have resource R available” or “I know fact F”), the second agent can prove from its zkGraph that the fact exists in its knowledge base (or prove a property about its state) without revealing the rest of its knowledge graph. In regulated DeFi, zkGraph might store customer credentials or trading history; a compliance verifier could query “does this address belong to a KYC-approved user?” and get a proof of the yes/no answer without seeing the user's identity itself. Essentially, zkGraph enables **information to be shared or checked with minimal exposure**.

For example, consider a **credit scoring asset**: Users' detailed financial data is stored in a zkGraph off-chain. A DeFi lending protocol queries this graph with “Is user X's credit score > 700?” The zkGraph layer allows the holder of the data to prove the answer (yes/no) is correct as per the committed data ²⁰, without revealing the actual credit report or score. The protocol thus only learns the minimal necessary info (whether the score passes the threshold) but gains **cryptographic assurance** of its validity.

Integration: zkGraph works closely with zkSyslog and zkRAG. Events from the log (zkSyslog) can trigger updates to the graph (ensuring the graph's state reflects what has happened). Conversely, when the reasoning layer (zkRAG) retrieves information, it will query zkGraph, and the retrieved facts come attached

with proofs of authenticity. This ensures the reasoning process only operates on trusted data. The combination of zkSyslog and zkGraph forms a complete **verifiable memory system**: the log provides temporal integrity of events, and the graph provides structured semantic memory, both **fully verifiable and privacy-preserving**.

*(Diagram – Verifiable Memory Illustration: Envision a diagram of a knowledge graph with nodes and edges. The entire graph has a committed root hash. A query (highlighted in a different color) runs on this graph (e.g. finding a path between two nodes); alongside the query result, a proof is output. The diagram shows the proof being checked against the graph's root commitment on-chain, verifying the query result is correct. All non-essential nodes/edges are not revealed to the verifier, indicated by them being greyed out or hidden.)**

zkRAG: Zero-Knowledge Retrieval-Augmented Reasoning Layer

Building on a trusted log and memory, the **zkRAG layer** is responsible for *reasoning* – making decisions or generating outputs (e.g. an AI agent answering a question or a strategy module deciding a trade) – in a manner that is **verifiably correct and accountable**. RAG (Retrieval-Augmented Generation) is a paradigm from AI where a model retrieves relevant context from a knowledge base and then produces an answer or action. In zkRAG, we infuse this process with zero-knowledge proof capabilities to create a **cognitive trust layer**. This means that not only do we expect the reasoning to be sound, but the agent can *prove* certain aspects of its reasoning to others without revealing its full thought process.

A zkRAG-enabled agent operates roughly in two phases: 1. **Retrieve**: The agent queries the knowledge base (the zkGraph or other data sources) for information relevant to the task. Because of zkGraph, the agent can provide **assurances of authenticity** for any retrieved data – for instance, it can prove “this snippet of data was indeed present in my knowledge base and up-to-date” (using inclusion proofs from the graph commitment). 2. **Generate (Reason)**: The agent then performs computation or reasoning using that information. In a classical LLM-based agent, this could be a chain-of-thought or prompt that leads to an answer. In a DeFi context, it could be an algorithm computing a recommendation. The zkRAG layer instruments this process such that **key steps are verifiable**. This might involve structuring the reasoning into verifiable computations (suitable for the HPEI layer to prove) or logging intermediate steps to zkSyslog with commitments.

The outcome is that the agent's **outputs or decisions come attached with proofs** that build trust. For example, an AI assistant might output an answer along with a ZK proof that “*I checked the following knowledge sources and they indeed support my answer*”. In regulated settings, an AI could prove it followed mandated guidelines (e.g. it consulted a compliance checklist from the knowledge graph) without revealing the entire checklist or its internal notes.

While fully verifying arbitrary reasoning of a neural network is extremely challenging (and an open research frontier), zkRAG focuses on *constrained or structured reasoning* where verification is tractable. This could include rule-based reasoning, numerical calculations, or flows that can be broken into sub-computations with known correctness criteria. It also includes **policy compliance proofs** – e.g. proving an AI's decision did not violate a rule. A blog on AI security suggests using ZKPs to achieve “*auditable AI decisions: demonstrating that an AI-driven decision followed a specific process without exposing the underlying logic.*”²¹. zkRAG is precisely about enabling this: the **verifier learns that the correct process was followed, but not the private details of that process**.

To illustrate, consider a **multi-step decision** by an agent: it might involve intermediate queries (to internet APIs, for example) and intermediate conclusions. The agent can log commitments to each step in zkSyslog. Using zkRAG, it can prove that *if* those steps were valid and led logically to the outcome. One concrete approach is using a **chain-of-thought commitment**: at each reasoning step, the agent records a cryptographic hash of its internal state. Later, if needed, it can reveal and prove consistency of certain steps. Researchers have proposed “*extending ZK proofs to track multi-step chains-of-thought so that incremental context expansions stay verifiable over time.*” ²². In practice, zkRAG might allow an agent to prove (in zero-knowledge) a statement like: “*I have a chain-of-thought of length N that results in conclusion Y, and each step of that chain is consistent with the data I retrieved,*” without revealing the chain itself. This would prevent, for example, an agent from shortcutting its reasoning or incorporating unauthorized data, because any deviation would break the proof consistency.

Cognitive Trust Example: Imagine a **DAO treasurer AI** that decides how to allocate funds. The DAO members require that any allocation decision *must be based on current budget data and adhere to a risk policy*. Using zkRAG, the AI can output a proposed allocation plus proofs that: (a) it retrieved the latest budget numbers from the zkGraph (so it isn't using stale or fake data), and (b) it simulated the allocation effect on risk metrics and those remained within policy thresholds (with that simulation done in a provable computation). The DAO gets high assurance that the decision is *grounded in truth and policy*, even though the AI's internal strategy (maybe an LLM reasoning about the narrative) remains confidential. This increases trust in autonomous decisions and could be critical for **verifiable AI systems in finance or governance**.

zkRAG essentially turns the black-box of AI or complex logic into a **gray-box** – not fully open, but with **selective transparency** where it matters for trust. By combining with zkSyslog and zkGraph, it draws upon verifiable data and logs; by combining with HPEI (below), it offloads heavy computation or formal proofs of logic to efficient ZK circuits. The net result is a layer of **reasoning accountability**. In multi-agent ecosystems, this allows agents to form higher-level assurances: e.g., an agent can enter into a smart contract with another agent that *requires a proof of proper reasoning for each action*, enabling cooperation under cryptographic guarantees. We will see more of this in the multi-agent use case discussions.

(Diagram – Verifiable Reasoning Flow: Depict an agent reasoning process with steps: retrieve data → think/compute → output action. Each retrieve step has an arrow to the knowledge base (graph) and returns data + a ZK proof. The think/compute step is instrumented by HPEI (maybe shown as a mini circuit icon) producing a proof of correctness of that computation or compliance check. The output action is delivered along with these proofs to a verifier (which could be another agent or an on-chain contract). The verifier icon checks the proofs and then accepts the action. This diagram highlights that between input and output, there are multiple verifiable checkpoints, even though the full internal reasoning is not exposed.)

HPEI: Hybrid Proof Execution Infrastructure (STARK+SNARK)

The **Hybrid Proof Execution Infrastructure (HPEI)** is the engine that powers the heavy lifting of UTS: it allows general computations (program logic, mathematical calculations, algorithmic decisions) to be executed with an accompanying proof of correctness, and it does so in a way that balances scalability and privacy by combining two leading ZK proof systems: **zk-STARKs** and **zk-SNARKs**.

- **STARKs (Scalable Transparent ARguments of Knowledge)** provide fast, massively scalable proof generation with no trusted setup, but tend to have larger proof sizes. They are well-suited for proving large computations (e.g. millions of cycles of a VM) and are post-quantum secure. However,

STARK proofs are transparent, meaning any secret data must be carefully handled (they don't natively hide inputs unless you commit via hash and prove consistency).

- **SNARKs (Succinct Non-Interactive ARGuments of Knowledge)**, especially modern SNARKs like Groth16 or PLONK, produce very small proofs and can more easily keep certain inputs private (since they can rely on cryptographic commitments internally). They often require a trusted setup and can be slower to generate for large circuits, but are efficient to verify.

HPEI **hybridizes these** by integrating STARK-based execution traces with SNARK-based proof compression and privacy shielding. In practice, this might look like running an **execution trace** of a program (or VM) in a STARK-friendly manner, then using a SNARK to either (a) prove knowledge of a secret that was input to the STARK trace, or (b) wrap the STARK proof in a much smaller proof for on-chain verification. For instance, an agent could execute a computation in a STARK VM (like StarkWare's Cairo VM or similar) to prove the *correctness of the computation*, while using SNARK sub-circuits to keep certain values hidden (like private keys, personal data, or proprietary model weights). The final result is a **composite proof** that attests to the overall statement (e.g., "Program X executed correctly on inputs Y and hidden input Z, yielding result R") with both the efficiency of STARKs and the succinctness/privacy of SNARKs.

This approach is supported by emerging research: in the context of zkEVMs (zero-knowledge Ethereum Virtual Machines), designs are exploring *"hybrid arithmetization that integrates STARK-style AIR constraints with a recursive SNARK proof wrapper."* This yields *"the transparency and parallelizable proving advantages of STARKs... together with the succinct verification efficiency of SNARKs."* ²³ . In other words, HPEI leverages STARKs for what they do best (fast proof of large computations) and SNARKs for what they excel at (tiny proofs and hiding secrets). One notable example is the RISC Zero system, which uses exactly this hybrid: a STARK-based CPU simulation wrapped in a SNARK to compress the proof ²³ .

Architecture: The HPEI can be thought of as a *provable virtual machine* or *zkVM* that UTS agents and protocols use to execute their code. When an agent needs to perform some logic (say a machine learning model inference, a financial calculation, or running an algorithm on private data), it sends this task to the HPEI. The HPEI might execute the task within a STARK environment (ensuring computational integrity) and simultaneously generate any necessary SNARK proofs for secret-dependent steps. These are then combined (often recursively) into a final proof artifact. Modern techniques like **proof recursion** allow multiple proofs (even of different types) to be merged: e.g. one could SNARK-verify a STARK proof inside another SNARK, or STARK-verify many SNARK proofs, etc., depending on efficiency needs.

By doing this, HPEI ensures that **any complex transformation or decision made off-chain can be independently verified on-chain**. This is crucial for trust in UTS: it's not enough to log events and store data; we must also trust the computations agents perform on that data. With HPEI, if an agent claims to have executed a particular algorithm with certain results, the claim comes with a proof. In a multi-agent contract, for example, an agent could be required to provide a valid HPEI proof for each action it takes (the on-chain logic would reject any action lacking a proof or with an invalid proof).

Privacy-Preserving Execution: A key benefit of combining SNARKs is that HPEI can prove statements like *"I executed algorithm A on input X (hidden) and got output Y"* without revealing X. For instance, an agent might run a proprietary trading strategy code on confidential market data; a STARK ensures the trade outcome is correctly computed, and a SNARK ensures the data remains secret yet was used consistently. This fulfills the promise of allowing external verification of computation *without exposing sensitive inputs*, aligning with the core idea of zero-knowledge proofs ⁴ .

Efficiency Considerations: STARK proofs, while larger, are practical to verify on modern L2s like Starknet because Starknet itself is built to handle STARK verification efficiently as part of its infrastructure. SNARK proofs are small and fast to verify even on Ethereum L1 (a few hundred bytes proof verified in a few milliseconds). HPEI ensures that what goes on-chain can be as minimal as needed. In fact, by verifying SNARKs inside STARKs or vice versa, one could optimize proof sizes. Some hybrid implementations report up to ~40% reduction in proof size through such combinations ²⁴ (though concrete numbers vary). The overarching goal is that **UTS participants do not worry about the cost of verification** – they can run heavy AI models or large computations off-chain and still have on-chain guarantees at low cost.

To cement this, Starknet has become an attractive platform for verifying external proofs: “You can now use Starknet to verify zk-SNARKs... Starknet is the cheapest way to publish zero-knowledge proofs.” ²⁵ . This is thanks to projects like **Garaga** which provide a Cairo implementation of SNARK verifiers ²⁶ , and the general efficiency of STARKs for batch verification. HPEI takes advantage of this by making Starknet not just a place to verify its own STARK, but a hub to verify any ZK proofs needed.

In summary, HPEI is the **execution layer of trust** – it turns arbitrary code execution into a verifiable act. Whether it's verifying a machine learning inference, a financial simulation, or a compliance check, HPEI produces proofs that feed into the UTS trust flow. This ensures that even the most complex off-chain logic can be treated with the same confidence as an on-chain smart contract, without actually running it on-chain.

*(Diagram – Hybrid Proof Execution: Imagine a flowchart of an execution: an input goes into a STARK Prover box (for heavy computation) producing an execution trace and a STARK proof. Meanwhile, secrets from the input go into a SNARK Prover box to prove some hidden predicate (or the SNARK takes the STARK proof as input to compress it). The outputs of both are combined into a final proof. That proof is sent to a Starknet verifier contract. The diagram highlights: STARK covers the whole computation trace, SNARK covers secret or compressing, and Starknet on-chain does the final verify.)**

On-Chain Verification on Starknet

The culmination of the UTS stack is **on-chain verification**, where all the proofs generated by the aforementioned layers are ultimately checked and enforced. Starknet, a Layer-2 network utilizing STARK proofs for consensus, acts as the **trust anchor** for UTS. By performing verification on-chain, we ensure that trust is not dependent on any single off-chain party – instead, the blockchain's security (decentralized and hard to subvert) backs the verification of all claims.

Role of On-Chain Logic: Smart contracts on Starknet (or an analogous ZK-enabled chain) are deployed to serve as **verifiers** and **adjudicators**. These contracts hold reference commitments (like the latest Merkle root of an agent's zkSyslog or zkGraph) and contain verification code for proofs (for both STARK and SNARK proofs, as needed). When an agent or protocol needs to assert something (e.g. an agent wants to take an on-chain action, or two agents want to settle a dispute), the proofs are submitted to these contracts. The contracts then:

- Check the validity of the proofs (using Starknet's crypto precompiles or Cairo implementations for hash checks, elliptic curve pairings for SNARKs, etc.).
- Check that the proofs attest to the correct statements relative to on-chain state. For example, a contract might verify a proof that “event E is in agent A's log” by checking the proof against agent A's

committed log root stored on-chain. If valid, the contract might trigger some state update (like releasing funds or marking a task complete). If invalid or absent, the contract can enforce penalties (like slashing stake or denying access).

This design enables **autonomous yet accountable agents**. For instance, a smart contract could hold an agent's bond and automatically slash it if a counterparty presents a proof of the agent's misbehavior (or if the agent fails to present a required proof in time) ¹⁵. The on-chain logic thus acts as an impartial judge: it doesn't need to know the details of the agent's internal processes, only that the cryptographic proofs check out ²⁷.

Starknet's choice is motivated by efficiency: verifying many proofs on Ethereum L1 directly would be expensive. Starknet, as a ZK-rollup, can aggregate the cost of verification into its STARK proof that is periodically posted to L1. Moreover, Starknet's architecture is designed with Cairo, a language amenable to ZK verification. It's feasible to implement complex verification algorithms (like pairing-based SNARK verifiers or Merkle proof checks) directly in Cairo and run them as needed. Starknet has already seen development of libraries to verify Noir/Circom proofs on-chain ²⁸ ²⁹.

Example: Suppose in a regulated DeFi scenario, a lending contract on Starknet integrates UTS. When a borrower agent tries to draw a loan, the contract requires: 1. A proof from zkKYC (via zkGraph) that the borrower is verified (as discussed earlier). 2. A proof from zkSyslog that the borrower agent hasn't exceeded certain actions (e.g., not opened multiple concurrent loans beyond a limit). 3. A proof from HPEI that the borrower's risk score was computed correctly from confidential data and exceeds a threshold.

Each of these proofs is checked by the on-chain verifier logic. Only if all pass does the contract execute the loan. If any proof fails, the loan is denied; if a proof is later found fraudulent (which should be impossible if cryptography holds, but imagine a dispute scenario), the contract could unwind the action or penalize accordingly. All this happens **trustlessly** – no off-chain authority is needed to approve the loan, yet the lender's requirements (KYC, risk, etc.) are satisfied in zero-knowledge.

From a security perspective, anchoring on-chain means **users don't need to trust the prover**. As long as the proofs verify on-chain, even if the agent itself were malicious, it cannot forge a proof for a false claim (that's the soundness of ZKPs). Thus, the on-chain verification is the ultimate **root of trust** in UTS, converting cryptographic assurances into enforceable outcomes (financial transactions, state changes, or penalties).

Finally, by placing verification on a public blockchain, UTS benefits from **transparency where it matters**: everyone can see that the proofs were accepted (or rejected), even if they cannot see the private data behind them. Regulators or participants can thus audit that certain rules were followed, purely by looking at on-chain proof verification events. This creates a powerful loop: **privacy for individual data, transparency for rule enforcement**. UTS achieves both via cryptography.

Use Cases and Enabled Innovations

The UTS stack unlocks a range of new possibilities by providing a *universal, composable trust layer*. Here we discuss several domains and examples where UTS can be transformative, focusing on new asset types and agent models that become feasible with provable execution, memory, and reasoning.

Provable Multi-Agent Systems and Autonomous Economies

In multi-agent systems – scenarios where multiple AI agents or autonomous processes interact, cooperate, or compete – trust and verification are critical. UTS enables **provably honest agents** and cryptoeconomic mechanisms to ensure cooperation. Consider a **decentralized agent economy** (like a network of AI services trading compute or information). Without UTS, an agent could claim it did some work or followed protocol, but others have to take that on faith. With UTS:

- Agents log their actions to zkSyslog and share commitments to those logs. If Agent A pays Agent B for a service, A can later demand a proof from B's log that B indeed executed the required steps (and B cannot fake this without it being caught via broken hashes).
- Agents can form smart contracts that require **proof-of-action**. For example, two agents enter a contract where each must perform certain tasks periodically; failure to submit a zk proof of the task results in automatic loss of a deposit. This was exemplified by Rodriguez's *"Zero-Knowledge Court"* concept: *"if another agent disputes your claims, the court verifies the proof and slashes your stake if you lied"* ³⁰. UTS provides the infrastructure for such a "court" – zkSyslog provides the evidence, HPEI/zkRAG provides the proofs of correct execution, and on-chain logic enforces penalties.

A **new agent model** emerges: the *Provable Autonomous Agent*. Such an agent isn't just a piece of AI software; it comes with an attached *cryptographic audit trail* and *bond* that makes it trustworthy in open environments. These agents can engage in long-running interactions (like contract negotiations, joint computations) knowing that cheating or deviating from agreed rules will be detectable via proofs ²². This greatly reduces the risk in multi-agent collaboration, leading to more stable coalitions and complex workflows delegated entirely to machines.

As a concrete example, imagine a **supply chain of AI agents** handling an e-commerce order (one agent handles payment, another shipping, another inventory). Using UTS, each agent can prove it performed its sub-task (payment processed, package sent, stock updated) without revealing internal API keys or sensitive info. If something goes wrong, the blame can be assigned by checking the proofs/logs. This level of accountability could give businesses confidence to automate multi-party workflows that today would require audits or intermediaries.

Regulated DeFi and Composable Compliance

Decentralized Finance often struggles with incorporating regulatory compliance (KYC/AML, risk limits, reporting) because traditional approaches would violate privacy or decentralization. UTS offers **composable compliance** – enforcement of rules through proofs instead of direct disclosure. This gives rise to **new asset types and financial models** that were previously impractical:

- **ZK-KYC Assets:** Tokens or instruments that can only be held or transacted by verified participants, without revealing their identities on-chain. Using zkGraph and HPEI, a user can prove "I am KYC-approved by authority X" when interacting with a DeFi protocol ²⁷. For example, a stablecoin could require proof-of-KYC for transfers above a threshold. Unlike current permissioned ledgers, this can be done without disclosing *who* the user is – only that they are approved. As described in a Studio AM analysis, *"the wallet uses the credential to generate a zero-knowledge proof that says, 'I have been verified by a trusted authority and meet the required criteria,' without revealing ... personal details. The protocol then performs a quick mathematical check... If the proof is valid, access is granted. This creates a*

secure ‘air gap,’ preserving privacy while immutably proving compliance on-chain.”²⁷ . UTS natively supports this flow through its layers (credential in zkGraph, proof via HPEI, check on-chain).

- **Provable Market Data and Oracles:** Assets that derive value from off-chain data (like derivatives, insurance, etc.) can use UTS to get tamper-proof oracles. An oracle agent can fetch data (e.g. a price feed or weather data), log the retrieval in zkSyslog, and then provide a zk proof that *given the data it fetched, it correctly computed the payout*. Because of the log and proof, users don’t have to trust the oracle’s word – any deviation would be provably detectable. This could enable **regulated derivatives** where the settlement is verifiably correct and based on approved data sources (which the oracle can prove it used, via retrieval proofs).
- **Composed Financial Contracts with AI:** UTS allows traditional financial contracts to include AI agents as components in a trustworthy way. For instance, consider an **investment fund token** managed by an AI strategy. Normally, giving an AI control over funds is risky due to lack of transparency. But with UTS, the AI’s actions (trades, reallocations) are all logged and proven. Investors could receive regular proofs that the AI adhered to the agreed strategy constraints (e.g. not more than 10% in a single asset, or only using whitelisted liquidity pools) – these constraints can be encoded and checked via zkRAG/HPEI. The fund token thus becomes a new kind of asset: **a provably compliant AI-managed fund**, combining human-level strategy flexibility with blockchain-level trust.
- **Privacy-Preserving Asset Transfers:** Using the layers, assets like securities or bonds could be traded on-chain while hiding sensitive details (like identity of holders or exact transaction amounts) but still proving compliance (e.g. total ownership doesn’t exceed a cap, all participants passed accreditation). UTS’s proof infrastructure can enforce such conditions globally without leaking specifics. This was nearly impossible before; now **institutional DeFi** can flourish, as institutions can participate knowing rules can be enforced by code rather than trusting anonymous counterparties

31 .

In summary, UTS paves the way for “**compliance by design**” in DeFi – rules become part of the asset’s and protocol’s logic. Market participants gain assurance (through automated proofs) that everyone is playing by the same rules, addressing regulators’ concerns while preserving user privacy and decentralization.

Verifiable AI Services and AI-Human Collaboration

As AI systems become more prevalent in critical roles (medical diagnosis, financial advising, content generation), a major challenge is how to **trust** these systems and integrate them into workflows that require accountability. UTS provides a path to **verifiable AI services**:

- **Proof-of-Integrity for AI Models:** With HPEI, one can generate proofs of model execution (as demonstrated by zkML projects proving GPT-2 inference⁶). A future AI service (say a cloud LLM API) could return not just an answer but a proof that *“this answer was indeed produced by running our audited model on your input, without alteration.”* This assures users that the model’s output is authentic (important for things like legal or scientific use of AI). It can also prove the model’s provenance (e.g., that a regulated model, not an unverified one, was used).

- **Safe AI Agents with Constraints:** By encoding certain constraints in zkRAG, we can have AI agents that **prove they followed safety rules**. For example, an AI content generator could prove it did not incorporate any copyrighted text beyond a permitted length from its training data. Or an AI could prove it checked a “do-not-answer” list for a question (ensuring it avoids disallowed advice) ³². These proofs might use zkGraph to represent forbidden knowledge and HPEI to scan the output against it in zero-knowledge, yielding a compliance proof. The result is AI that can be more freely used in sensitive contexts because it provides guarantees of adherence to policies (useful for enterprise AI deployment, where compliance and audit trails are mandatory).
- **Multi-Agent AI Systems (MAS) with Verifiable Cooperation:** In advanced setups like a team of AI assistants working on a project, UTS enables them to **trust but verify** each other's contributions. Agents can specialize (research agent, coding agent, validation agent) and exchange results with attached proofs. A validation agent could even use UTS to probabilistically verify another agent's work (e.g., check a few log entries or re-run a part of the computation in ZK for consistency). This minimizes the chance of one rogue agent polluting the overall outcome.
- **Human-AI Collaboration:** From a user's perspective, imagine interacting with an AI advisor that not only provides advice but can *prove certain qualities* of that advice. A medical AI could prove it cross-referenced the latest approved treatment guidelines (so the doctor trusts it didn't hallucinate something outside standard of care). An AI tutor could prove it has a degree of mastery (e.g., by proving it can solve benchmark problems in ZK). While some of these ideas are forward-looking, UTS lays the groundwork by treating reasoning and knowledge as provable commodities.

The overall effect is that AI systems no longer have to be black boxes that we either blindly trust or constantly supervise. We gain a spectrum of verifiable assurances that can be tailored per application. This can accelerate adoption of AI in regulated industries (finance, healthcare, government) where lack of trust has been a barrier.

New Asset Classes and Digital Interactions

Lastly, beyond the main domains above, UTS opens doors to creative new asset classes and interactions at the intersection of blockchains and off-chain worlds:

- **Data Assets with Usage Control:** Data could be turned into an asset where each access or usage of the data is logged via zkSyslog and only permitted if a proof of payment (or other condition) is provided. For example, a dataset NFT could require any AI training on it to produce a proof (via HPEI) that a usage fee was paid and the training followed the licensed usage limits. The zkGraph could store licensing terms, zkSyslog logs model training events, and on-chain logic mediates payment. This would enforce digital rights in a way previously reliant on trust.
- **Dynamic NFTs and Soulbound Tokens:** NFTs or soulbound tokens (non-transferable badges) can embed zk proofs of characteristics. For instance, a soulbound token might represent a diploma where the issuer (a university) has proven the holder passed certain exams (without exposing grades) and on-chain anyone can verify the diploma's authenticity. Or an NFT avatar in a game might carry a zk proof that it achieved a certain off-chain skill (preventing spoofing of in-game achievements). UTS would handle the off-chain proof generation (e.g. the game logs and verifies the achievement using zkSyslog/HPEI, then issues an NFT with that proof attached).

- **Provable IoT and Real-World Linkages:** In use-cases like supply chain or environmental assets (carbon credits, etc.), IoT sensors or oracles could feed data that must be trustworthy. UTS could log sensor readings in zkSyslog and later prove a summary or threshold condition. A carbon credit token, for example, might come with proofs that *“the associated project reduced X tons of CO2 as measured by sensor network Y over period Z”*. Each sensor's data is private, but the aggregate claim is proven from those private logs. This turns real-world events into verifiable digital claims, enhancing the credibility of tokenized real assets.

Across these examples, **composability** is key. Because UTS standardizes the interface of trust (mostly through proofs and on-chain verifiers), disparate systems can mix and match these capabilities. A DeFi contract can plug into an AI oracle service, an AI agent can leverage a DeFi identity credential, and so on – all mediated by universally verifiable proofs. This composability at the trust layer is akin to how *smart contracts allowed composability at the state/logic layer* in Ethereum. UTS could usher in a similar wave of innovation by making *trust and verification a shared resource* rather than an application-specific burden.

Conclusion: UTS and the Future of Trust in Autonomous Systems

The Universal Trust Substrate as described – an integrated stack of zkSyslog, zkGraph, zkRAG, HPEI, and on-chain verification – represents a **paradigm shift in how we establish trust in digital interactions**. By weaving together cryptographic proofs with system design, UTS enables a world where **“verification is native”**: every action, decision, or data exchange can carry its own evidence of integrity and compliance.

This has profound implications for the future:

- **Multi-Agent Systems:** UTS can become the backbone of *machine economies* and AI ecosystems. It provides the guarantees needed for independent agents to safely cooperate and transact. In the same way TCP/IP provided a universal substrate for communication between computers, UTS could be the substrate that allows autonomous agents (with different owners, objectives, even legal jurisdictions) to interact with **built-in trust**. This will accelerate the development of complex, decentralized AI networks – from smart cities with autonomous services to global agent marketplaces – because the fear of betrayal or error can be mitigated by verifiable proofs. We expect new forms of organization (DAOs of AIs, or human-AI hybrid firms) to flourish on top of UTS, enjoying both autonomy and oversight.
- **Regulated DeFi and Institutional Adoption:** UTS strikes at the heart of the **compliance vs. decentralization** impasse. It shows that these need not be opposites: one can have privacy and decentralization while still satisfying regulatory requirements through zero-knowledge verification. This opens the door for institutional capital and traditional assets to migrate on-chain, as well as for regulators to more comfortably approve decentralized platforms. Financial contracts can be self-enforcing not just in terms of payments, but in terms of rules and conditions. Over time, we may see a financial system emerge that is global and automated, yet provably fair and compliant – something that would be unimaginable without technologies like UTS.
- **Verifiable AI and AI Safety:** Perhaps most far-reaching, UTS offers a toolset for aligning and auditing AI systems. In discussions about AI alignment and safety, a key challenge is how to ensure an AI acts as intended without constant supervision. UTS provides a mechanism for *accountability*: AI behaviors can be logged, decisions can be forced through verifiable checkpoints, and misuse can be

detected and proven. This won't solve all AI safety problems, but it adds a powerful component to the toolbox: **cryptographic truth** about what an AI did or didn't do. As AI systems become more agentic and are entrusted with more authority, such a substrate for trust will be critical to integrate them safely into society and the economy.

In conclusion, the Universal Trust Substrate is more than the sum of its parts. It is a **unifying framework for trust** across traditionally disparate realms – finance, data, AI, and beyond. By standardizing how we prove and verify claims, UTS can render transparency and privacy as compatible goals, rather than trade-offs. It positions itself as a *universal middleware* for a future where **transactions are not just financial** but also informational and cognitive. Those transactions – be it two AIs bargaining, a person delegating decisions to an AI, or a bank executing a contract – can all share a common foundation of provable trust.

The thesis driving UTS is that **provable integrity will become as fundamental as connectivity** in the infrastructure of tomorrow's digital world. Just as the internet connected billions of agents (human or machine) in a common network, the Universal Trust Substrate aspires to *secure* those connections with a common fabric of verifiability. In doing so, it can unlock unprecedented levels of automation, collaboration, and innovation, safe in the knowledge that even as systems grow more complex and autonomous, **the truth of their operations is always accessible and assured**.

Ultimately, UTS charts a path toward a future where we can confidently delegate to algorithms, scale decentralized organizations, and interconnect systems of record with systems of intelligence – all without sacrificing control or trust. It is a bet that **the combinatorial power of composability, privacy-preserving proofs, and smart contract enforcement** will form the pillar of **the next generation of digital infrastructure** – one where “*Don't trust, verify*” is not just a mantra, but an everyday reality woven seamlessly into the substrate of our multi-agent, DeFi, and AI-driven world. ¹ ²⁷

¹ ² ³ ¹³ ¹⁴ ¹⁵ ¹⁶ ²² ³⁰ Cryptoeconomic Justice: A Zero-Knowledge Court for LLM Agents | by Daniel Rodríguez | sadasant | Medium
<https://medium.com/sadasant/cryptoeconomic-justice-a-zero-knowledge-court-for-llm-agents-0bf7cf7196e3>

⁴ Zero-knowledge proof - Wikipedia
https://en.wikipedia.org/wiki/Zero-knowledge_proof

⁵ ¹⁷ ¹⁸ ¹⁹ ²⁰ Zero-Knowledge Verifiable Graph Query Evaluation via Expansion-Centric Operator Decomposition
<https://arxiv.org/pdf/2507.00427>

⁶ Lagrange: DeepProve-1: The First zkML System to Prove a Full LLM Inference
<https://lagrange.dev/blog/deepprove-1>

⁷ ⁸ ⁹ ¹⁰ ¹¹ ¹² Trustworthy AI Agents: Verifiable Audit Logs
<https://www.sakurasky.com/blog/missing-primitives-for-trustworthy-ai-part-5/>

²¹ ³² Leveraging Zero-Knowledge Proofs in Machine Learning | CSA
<https://cloudsecurityalliance.org/blog/2024/09/20/leveraging-zero-knowledge-proofs-in-machine-learning-and-llms-enhancing-privacy-and-security>

²³ ²⁴ Constraint-Level Design of zkEVMs: Architectures, Trade-offs, and Evolution
<https://arxiv.org/pdf/2510.05376>

25 26 Zero-Knowledge Magic for Cheap. Prove you know without telling me the... | by Tekkac | Medium
<https://medium.com/@tekkac/zero-knowledge-magic-on-starknet-56ac2453234f>

27 31 ZK-KYC: The Future of Institutional Compliance in Decentralized Finance – Studio AM | Official Website
<https://studioam.ai/zk-kyc-the-future-of-institutional-compliance-in-decentralized-finance/>

28 Verify Noir ZK proofs on Starknet with Garaga SDK
<https://www.starknet.io/blog/noir-on-starknet/>

29 Introduction - Starknet Documentation
https://docs.starknet.io/learn/S-two-book/how-it-works/stark_proof